FIELD GUIDE · MODEL CONTEXT PROTOCOL · READ IN ORDER

What is MCP, and how does it actually work?

A language model is brilliant and completely sealed off. It cannot open your calendar, read your repo, or touch your database. **MCP is the socket that lets it reach out** — one standard shape, so nobody has to hand-wire every app to every tool ever again.

I The tangle before it

II Two analogies

III Anatomy of a connection

IV The six beats

V Monday, 9:12 a.m.

VI What a server offers

VII What bites people

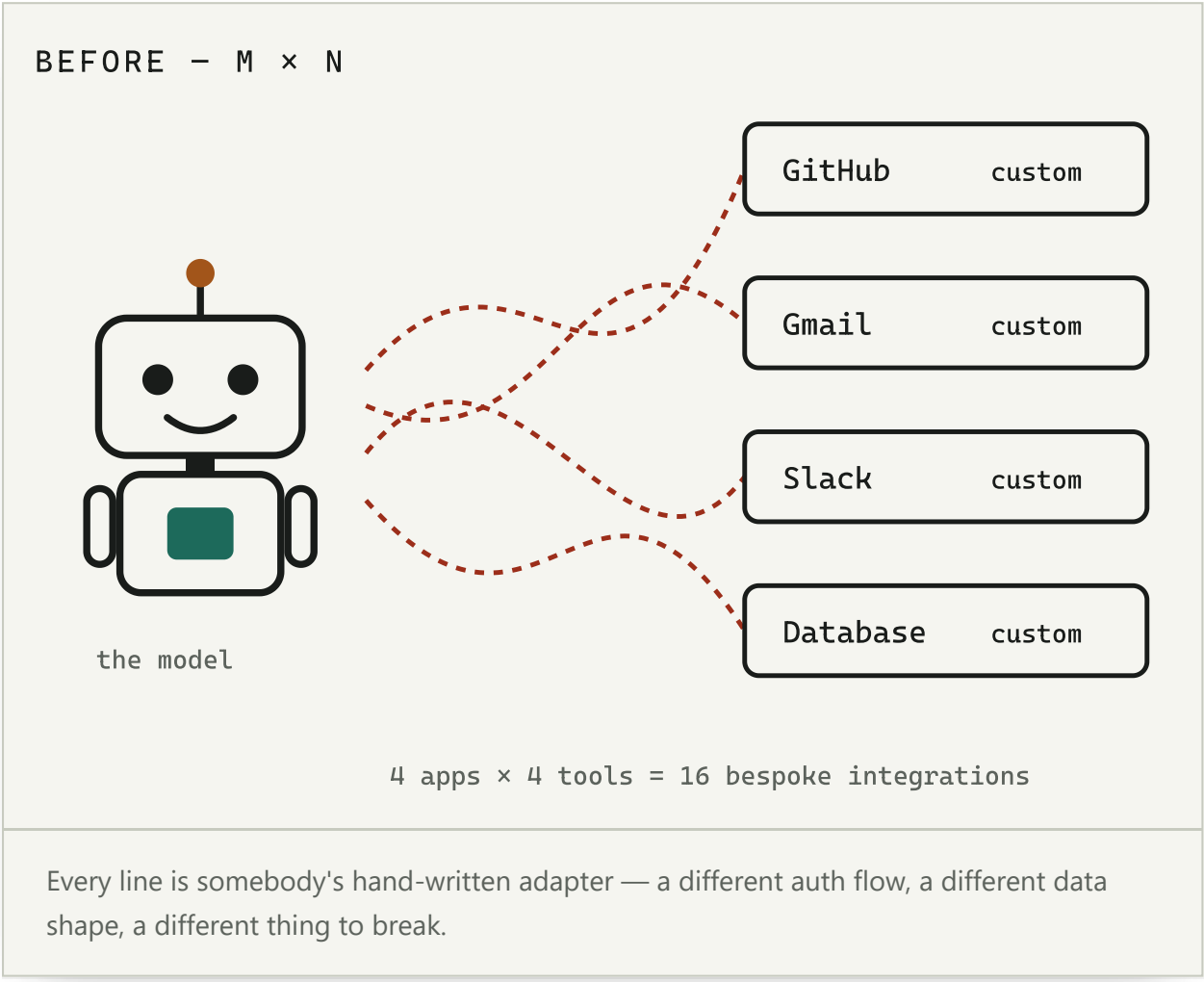
IN ONE LINE

MCP is a standard way for an AI app to **discover** and **use** outside tools and data — so nobody has to hand-write a custom integration for every app-to-tool pair. Write the server once; every MCP app on earth can use it.

PLATE I

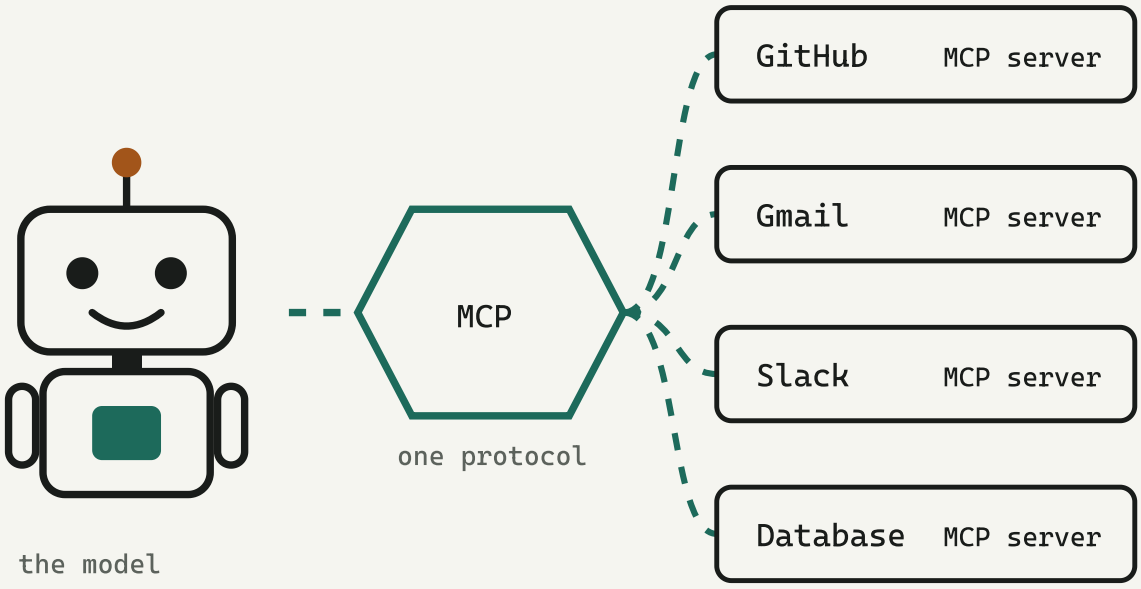
Before MCP: every connection was hand-soldered

Want Claude to read GitHub? Somebody writes a GitHub integration. Want Cursor to read GitHub too? Somebody writes it *again*. Four apps × four tools is sixteen bespoke pieces of code, each one someone's problem to maintain. This is the **M×N problem**, and it's the whole reason MCP exists.



Every line is somebody's hand-written adapter — a different auth flow, a different data shape, a different thing to break.

AFTER — M + N



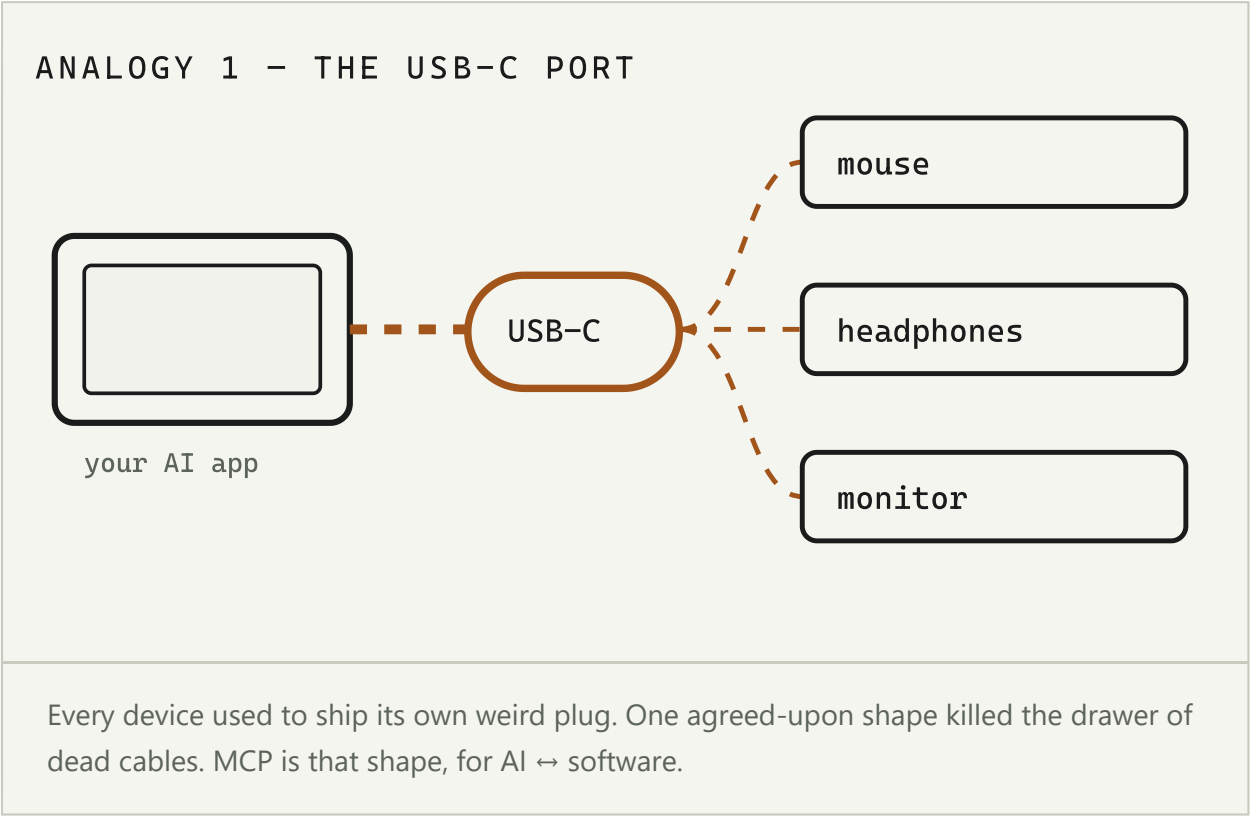
learn one plug shape → reach everything

Same four tools, but each is now written *once* against a published protocol — and any MCP-speaking app inherits all of them for free.

PLATE II

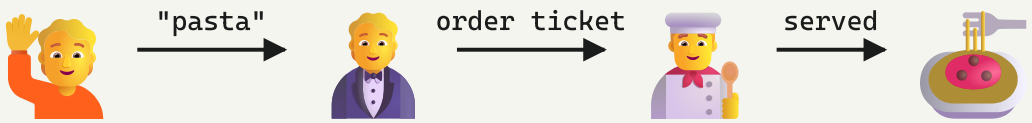
Two analogies, and the second one is the useful one

The first explains *why* MCP exists. The second explains *how it behaves at runtime* — which is the part people actually get wrong.



Every device used to ship its own weird plug. One agreed-upon shape killed the drawer of dead cables. MCP is that shape, for AI ↔ software.

ANALOGY 2 — THE RESTAURANT



YOU	WAITER	KITCHEN	RESULT
the user	= MCP itself	= the MCP server	back to you

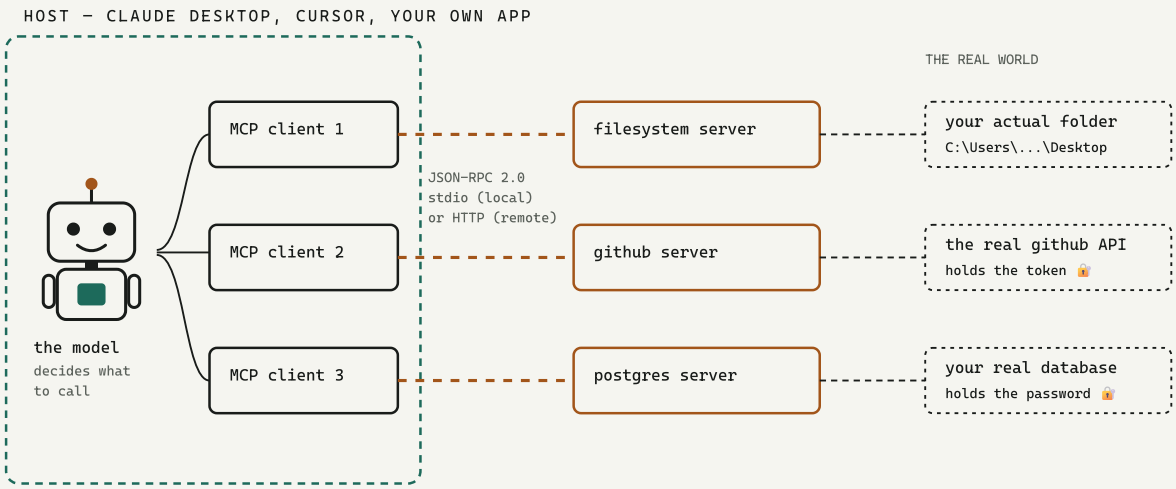
You never walk into the kitchen. You never learn the oven.
The waiter carries a MENU — that menu is the list of tools —
and one standard way to place any order on it.

Hold on to this one. The waiter carries requests and results; he does not decide what you're hungry for. Neither does MCP.

PLATE III

Anatomy of a connection

Four roles, and they're constantly mixed up. The **host** is the app you're looking at. Inside it, one **client** per connection. Each client is wired to exactly one **server**, and the server is the only thing that ever touches **the real system** — and the only thing holding a credential.



One client ↔ one server, always paired. The server is a deliberately dumb bridge: it advertises what it can do and does it on request. It never reasons. The model reasons.

PLATE IV

The six beats of a single request

This is the entire protocol in motion. Left column: what happens in human terms.
Right column: roughly what goes over the wire.

01 Handshake

The app boots and opens a connection. Both sides state who they are and what they support — version, capabilities, nothing else.

```
→ "method": "initialize"
← { "protocolVersion": "…",
    "capabilities": { "tools": {} } }
```

02 Read the menu

The client asks what the server can do. The reply is a list of tools with names, descriptions and typed arguments — this is what the model will read.

```
→ "method": "tools/list"
← [ "search_inbox",
    "send_email" ]
```

03 You ask for something

In plain words. You never name a tool, never see a schema, never touch the plumbing.

```
"Email my boss that I'll
 be late today."
```

04 The model chooses

It matches your sentence against the menu, picks one tool, and fills in the arguments. This is the only step involving any intelligence.

```
{ "tool": "send_email",
  "arguments": {
    "to": "boss@acme.com",
    "subject": "Running late" } }
```

05 You approve, the server acts

The host stops and asks. Only after you allow it does the server call the real API — a real side effect in the real world.

```
→ "method": "tools/call"
   …awaiting your approval…
   POST https://gmail.googleapis.com/…
```

06 The result comes home

Raw output goes back into the conversation, and the model turns it into a sentence a human wants to read.

```
← { "status": "sent", "id": "18f2" }

"Sent — your boss knows. ✅"
```

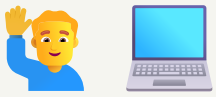
PLATE V

Monday, 9:12 a.m. — one question, end to end

The same six beats, but as they actually feel from the outside: four seconds and one sentence.

1 · 9:12 A.M., YOU

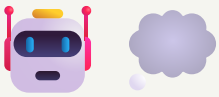
Claude — what's my first meeting today, and who's in it?



No API. No code. No switching tabs. **Beat 03.**

2 · THE MODEL THINKS

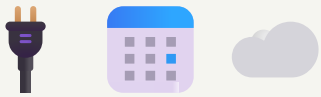
On my own I can't see his calendar… but a Calendar server is plugged in, and it has `get_events`.



Matching the request to the menu. **Beat 04.**

3 · THE SERVER WORKS

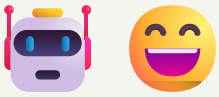
`tools/call` → the real Google Calendar API → `{ 09:30, "Design review", ["Sara","Ali"] }`



Only the server holds the key — Claude never sees your password. **Beat 05.**

4 · 9:12 A.M., ANSWERED

09:30 — Design review with Sara and Ali. Want me to email them the agenda?



Want that email sent too? Plug in a Gmail server. Same protocol, no new code. **Beat 06.**

PLATE VI

What a server can offer you

Three kinds of thing, and the useful distinction between them is *who decides when it's used*.

Tools

THE MODEL PICKS THESE

Verbs. Things that *do* something, usually with a consequence — which is why you get an approval prompt.

- `send_email`
- `run_sql`
- `create_issue`
- `book_flight`

Resources

THE APP HANDS THESE OVER

Nouns. Things to *read*. Pulled into the conversation as reference material, like quietly attaching a document.

- `a file on disk`
- `a database row`
- `a PDF`
- `yesterday's logs`

Prompts

YOU TRIGGER THESE

Templates — a canned expert workflow the server's author wrote for you. They surface as commands you fire on purpose.

- `/review-pr`
- `/summarize-standup`
- `/explain-error`

PLATE VII

What bites people

Nothing here is exotic. All three are things people learn the week after they set up their first server.

A server runs code on your machine

Installing one is closer to installing a program than adding a browser bookmark. Read what it does and who wrote it before you plug it in.

Eighty tools is not better than eight

Every tool description competes for the model's attention. Connect the servers you actually use; a bloated menu makes worse choices, not more capable ones.

MCP is the wiring, not the brain

The protocol carries requests and results. It doesn't make the model smarter, and it won't rescue a vague instruction. Judgment stays with the model — and with you at the approval prompt.

▼ GLOSSARY — THE WORDS PEOPLE THROW AROUND

`host`	The app you're actually looking at: Claude Desktop, Cursor, VS Code, or something you built.
`client`	The piece inside the host that maintains one connection. One client per server, always.
`server`	The small bridge program that exposes a system — your files, GitHub, a database — as tools.
`transport`	How the two talk: stdio for a server running on your machine, HTTP for a remote one.
`JSON-RPC 2.0`	The message format underneath. Boring on purpose, and already understood by everything.
`tools/list`	The call that fetches the menu. Everything the model knows how to do here came from this reply.
`tools/call`	The call that actually runs one. This is the one that changes the world, so this is the one you approve.
